WetInk Next

Полный исходный код, созданный в P0 - P3

- Включены исходники Kotlin, тесты, Android-ресурсы, Gradle-конфигурация, CI и wrapper scripts.
- Не включены бинарный Gradle Wrapper JAR, build-артефакты, локальные SDK-настройки и пустые .gitkeep.
- Файлов в листинге: 35.

.github/workflows/android.yml

```yaml
name: Android CI

on:
  push:
  pull_request:

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - uses: android-actions/setup-android@v3
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew :app:compileDebugKotlin :app:testDebugUnitTest :app:lintDebug
```

.gitignore

```
.gradle/
build/
local.properties
*.iml
.idea/
**/build/
```

settings.gradle.kts

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "WetInk Next"
include(":app")
```

build.gradle.kts

```
plugins {
    id("com.android.application") version "8.6.1" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
}
```

gradle.properties

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
android.useAndroidX=true
android.overridePathCheck=true
kotlin.code.style=official
org.gradle.daemon=true
org.gradle.caching=true
```

gradle/wrapper/gradle-wrapper.properties

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

gradlew

```
#!/bin/sh

APP_HOME=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd -P) || exit 1
JAVA_CMD="${JAVA_HOME:+$JAVA_HOME/bin/}java"
exec "$JAVA_CMD" -classpath "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain "$@"
```

```
@rem Minimal Gradle startup script for Windows.
@echo off
setlocal
set APP_HOME=%~dp0
if defined JAVA_HOME goto javaHome
java -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
goto end
:javaHome
"%JAVA_HOME%\bin\java.exe" -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
:end
```

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
}

android {
    namespace = "com.wetinknext"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.wetinknext"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "0.1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }

    buildFeatures {
        compose = true
    }

}

dependencies {
    val composeBom = platform("androidx.compose:compose-bom:2024.09.03")
    implementation(composeBom)
    androidTestImplementation(composeBom)

    implementation("androidx.core:core-ktx:1.13.1")
    implementation("androidx.activity:activity-compose:1.9.2")
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")

    testImplementation("junit:junit:4.13.2")

    debugImplementation("androidx.compose.ui:ui-tooling")
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

app/src/main/AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/Theme.WetInkNext">
        <activity
            android:name=".app.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

app/src/main/res/values/strings.xml

```xml
<resources>
    <string name="app_name">WetInk Next</string>
</resources>
```

app/src/main/res/values/themes.xml

```
<resources>
    <style name="Theme.WetInkNext" parent="android:style/Theme.Material.NoActionBar" />
</resources>
```

```kotlin
package com.wetinknext.app

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import com.wetinknext.engine.core.PaintSurfaceView
import com.wetinknext.ui.theme.WetInkNextPalette

@Composable
fun EditorScreen() {
    val palette = WetInkNextPalette.default

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(palette.appBg),
    ) {
        AndroidView(
            modifier = Modifier.fillMaxSize(),
            factory = { context -> PaintSurfaceView(context) },
        )

        Row(
            modifier = Modifier
                .align(Alignment.TopEnd)
                .padding(top = 12.dp, end = 12.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(24.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(24.dp))
                .padding(horizontal = 10.dp, vertical = 8.dp),
            horizontalArrangement = Arrangement.spacedBy(10.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            repeat(7) {
                Box(Modifier.size(18.dp).background(palette.textPrimary.copy(alpha = 0.9f), CircleShape))
            }
            Box(Modifier.size(26.dp).background(palette.accent, CircleShape))
        }

        Column(
            modifier = Modifier
                .align(Alignment.CenterStart)
                .padding(start = 12.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(24.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(24.dp))
                .padding(horizontal = 10.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            repeat(5) {
                Box(Modifier.size(22.dp).background(palette.textPrimary.copy(alpha = 0.9f), CircleShape))
            }
        }

        Column(
            modifier = Modifier
                .align(Alignment.BottomCenter)
                .padding(bottom = 20.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(28.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(28.dp))
                .padding(horizontal = 18.dp, vertical = 12.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            Text("WetInk Next", color = palette.textPrimary, fontSize = 16.sp)
            Text("P0: app boots, surface attached", color = palette.textSecondary, fontSize = 12.sp)
        }
    }
}
```

```kotlin
package com.wetinknext.app

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { EditorScreen() }
    }
}
```

```kotlin
package com.wetinknext.engine.core

import java.util.concurrent.atomic.AtomicReference
import kotlin.math.min

/** Thread-safe camera state shared by the UI and GL threads. */
class Camera(initial: ViewTransform = ViewTransform()) {
    private val state = AtomicReference(initial)

    fun snapshot(): ViewTransform = state.get()

    fun set(transform: ViewTransform) {
        state.set(transform)
    }

    fun update(transform: (ViewTransform) -> ViewTransform) {
        while (true) {
            val current = state.get()
            if (state.compareAndSet(current, transform(current))) return
        }
    }

    fun reset() {
        state.set(ViewTransform())
    }

    fun fitCanvas(
        canvasWidth: Int,
        canvasHeight: Int,
        viewWidth: Int,
        viewHeight: Int,
        marginRatio: Float = 0.04f,
    ) {
        if (canvasWidth <= 0 || canvasHeight <= 0 || viewWidth <= 0 || viewHeight <= 0) return
        val usableWidth = viewWidth * (1f - marginRatio * 2f)
        val usableHeight = viewHeight * (1f - marginRatio * 2f)
        val scale = min(usableWidth / canvasWidth, usableHeight / canvasHeight)
            .coerceIn(ViewTransform.MIN_SCALE, ViewTransform.MAX_SCALE)
        state.set(
            ViewTransform(
                scale = scale,
                translateX = (viewWidth - canvasWidth * scale) * 0.5f,
                translateY = (viewHeight - canvasHeight * scale) * 0.5f,
            ),
        )
    }
}
```

```kotlin
package com.wetinknext.engine.core

import kotlin.math.ceil
import kotlin.math.floor
import kotlin.math.max
import kotlin.math.min

/** Reusable mutable dirty rectangle in canvas coordinates. */
class DirtyRect {
    var left = 0f; private set
    var top = 0f; private set
    var right = 0f; private set
    var bottom = 0f; private set
    var isEmpty = true; private set

    fun clear() {
        left = 0f; top = 0f; right = 0f; bottom = 0f; isEmpty = true
    }

    fun include(x: Float, y: Float, radius: Float) = include(x - radius, y - radius, x + radius, y + radius)

    fun include(left: Float, top: Float, right: Float, bottom: Float) {
        if (right < left || bottom < top) return
        if (isEmpty) {
            this.left = left; this.top = top; this.right = right; this.bottom = bottom; isEmpty = false
            return
        }
        this.left = min(this.left, left); this.top = min(this.top, top)
        this.right = max(this.right, right); this.bottom = max(this.bottom, bottom)
    }

    fun include(other: DirtyRect) {
        if (!other.isEmpty) include(other.left, other.top, other.right, other.bottom)
    }

    fun expand(pixels: Float) {
        if (!isEmpty && pixels > 0f) {
            left -= pixels; top -= pixels; right += pixels; bottom += pixels
        }
    }

    fun clamp(canvasWidth: Float, canvasHeight: Float) {
        if (isEmpty) return
        left = left.coerceIn(0f, canvasWidth); top = top.coerceIn(0f, canvasHeight)
        right = right.coerceIn(0f, canvasWidth); bottom = bottom.coerceIn(0f, canvasHeight)
        if (right <= left || bottom <= top) clear()
    }

    fun copyTo(out: DirtyRect) {
        out.left = left; out.top = top; out.right = right; out.bottom = bottom; out.isEmpty = isEmpty
    }

    fun toPixelBounds(out: IntArray) {
        require(out.size >= 4)
        if (isEmpty) {
            out[0] = 0; out[1] = 0; out[2] = 0; out[3] = 0
            return
        }
        out[0] = floor(left).toInt(); out[1] = floor(top).toInt()
        out[2] = ceil(right).toInt(); out[3] = ceil(bottom).toInt()
    }
}
```

```kotlin
package com.wetinknext.engine.core

import android.opengl.GLES30
import android.opengl.GLSurfaceView
import com.wetinknext.engine.gl.CanvasGeometry
import com.wetinknext.engine.gl.GlCaps
import com.wetinknext.engine.gl.GlCheck
import com.wetinknext.engine.gl.GlProgram
import com.wetinknext.engine.gl.ShaderLib
import javax.microedition.khronos.egl.EGLConfig
import javax.microedition.khronos.opengles.GL10

class EngineRenderer(
    private val documentWidth: Int = DEFAULT_CANVAS_WIDTH,
    private val documentHeight: Int = DEFAULT_CANVAS_HEIGHT,
) : GLSurfaceView.Renderer {
    private val paintEngine = PaintEngine()
    private var caps: GlCaps? = null
    private var presentProgram: GlProgram? = null
    private var geometry: CanvasGeometry? = null
    private var screenWidth = 1
    private var screenHeight = 1
    private var uTexture = -1
    private var uCanvasToClip = -1
    private var uCanvasSize = -1
    private val presentMatrix = FloatArray(16)

    override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
        releaseGlObjects()
        val nextCaps = GlCaps.query(); caps = nextCaps
        presentProgram = GlProgram(ShaderLib.canvasPresentVertex, ShaderLib.presentFragment)
        paintEngine.create(nextCaps, documentWidth, documentHeight)
        paintEngine.clearCanvas()
        geometry = CanvasGeometry().also { it.create(documentWidth, documentHeight) }
        presentProgram!!.use()
        uTexture = GLES30.glGetUniformLocation(presentProgram!!.id, "uTexture")
        uCanvasToClip = GLES30.glGetUniformLocation(presentProgram!!.id, "uCanvasToClip")
        uCanvasSize = GLES30.glGetUniformLocation(presentProgram!!.id, "uCanvasSize")
        check(uTexture >= 0 && uCanvasToClip >= 0 && uCanvasSize >= 0) { "P3 present shader uniforms missing" }
        GlCheck.noError("P3 surface creation")
    }

    override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
        screenWidth = width.coerceAtLeast(1); screenHeight = height.coerceAtLeast(1)
        GLES30.glViewport(0, 0, screenWidth, screenHeight)
        paintEngine.camera.fitCanvas(paintEngine.canvasWidth, paintEngine.canvasHeight, screenWidth, screenHeight)
        GlCheck.noError("P3 surface changed")
    }

    override fun onDrawFrame(gl: GL10?) {
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
        GLES30.glViewport(0, 0, screenWidth, screenHeight)
        GLES30.glDisable(GLES30.GL_BLEND); GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
        GLES30.glClearColor(0.08f, 0.09f, 0.12f, 1f); GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
        val program = presentProgram ?: return
        program.use()
        paintEngine.camera.snapshot().buildCanvasToClip(screenWidth.toFloat(), screenHeight.toFloat(), presentMatrix)
        GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, presentMatrix, 0)
        GLES30.glUniform2f(uCanvasSize, paintEngine.canvasWidth.toFloat(), paintEngine.canvasHeight.toFloat())
        GLES30.glActiveTexture(GLES30.GL_TEXTURE0); GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, paintEngine.canvasTarget.textureId)
        GLES30.glUniform1i(uTexture, 0); geometry?.draw(); GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
    }

    fun cancelActiveStroke() = Unit
    fun releaseGlObjects() {
        geometry?.release(); geometry = null
        presentProgram?.release(); presentProgram = null
        paintEngine.release(); caps = null
        uTexture = -1; uCanvasToClip = -1; uCanvasSize = -1
    }

    companion object { const val DEFAULT_CANVAS_WIDTH = 1500; const val DEFAULT_CANVAS_HEIGHT = 2000 }
}
```

```kotlin
package com.wetinknext.engine.core

import com.wetinknext.engine.gl.GlCaps
import com.wetinknext.engine.gl.RenderTarget

/** Document state that is exclusively owned by the GLSurfaceView render thread. */
class PaintEngine {
    val camera = Camera()
    val canvasTarget = RenderTarget()
    var canvasWidth = 1; private set
    var canvasHeight = 1; private set
    var initialized = false; private set

    fun create(caps: GlCaps, width: Int, height: Int) {
        require(width > 0 && height > 0)
        release()
        canvasWidth = width; canvasHeight = height
        val useHalfFloat = caps.supportsHalfFloatColorBuffer && canvasTarget.probeHalfFloatColorBuffer()
        canvasTarget.create(width, height, useHalfFloat)
        initialized = true
    }

    fun resize(caps: GlCaps, width: Int, height: Int) {
        if (!initialized || canvasWidth != width || canvasHeight != height) create(caps, width, height)
    }

    fun clearCanvas() {
        check(initialized)
        canvasTarget.clear(1f, 1f, 1f, 1f)
    }

    fun release() {
        canvasTarget.release(); initialized = false; canvasWidth = 1; canvasHeight = 1
    }
}
```

```kotlin
package com.wetinknext.engine.core

import android.content.Context
import android.opengl.GLSurfaceView
import android.util.AttributeSet

class PaintSurfaceView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null,
) : GLSurfaceView(context, attrs) {
    private val engineRenderer = EngineRenderer()

    init {
        setEGLContextClientVersion(3)
        setRenderer(engineRenderer)
        renderMode = RENDERMODE_CONTINUOUSLY
    }

    override fun onPause() {
        engineRenderer.cancelActiveStroke()
        super.onPause()
    }

    override fun onDetachedFromWindow() {
        queueEvent { engineRenderer.releaseGlObjects() }
        super.onDetachedFromWindow()
    }
}
```

```kotlin
package com.wetinknext.engine.core

import kotlin.math.abs
import kotlin.math.cos
import kotlin.math.sin

/** Canonical canvas-to-view-local-screen transformation. */
data class ViewTransform(
    val translateX: Float = 0f,
    val translateY: Float = 0f,
    val scale: Float = 1f,
    val rotationRad: Float = 0f,
    val flipX: Boolean = false,
) {
    private val flipSign: Float
        get() = if (flipX) -1f else 1f

    val m00: Float get() = cos(rotationRad) * scale * flipSign
    val m01: Float get() = -sin(rotationRad) * scale
    val m10: Float get() = sin(rotationRad) * scale * flipSign
    val m11: Float get() = cos(rotationRad) * scale

    fun canvasToScreen(canvasX: Float, canvasY: Float, out: FloatArray) {
        require(out.size >= 2)
        out[0] = m00 * canvasX + m01 * canvasY + translateX
        out[1] = m10 * canvasX + m11 * canvasY + translateY
    }

    fun screenToCanvas(screenX: Float, screenY: Float, out: FloatArray) {
        require(out.size >= 2)
        val determinant = m00 * m11 - m01 * m10
        if (abs(determinant) < 1e-8f) {
            out[0] = 0f
            out[1] = 0f
            return
        }
        val inverseDeterminant = 1f / determinant
        val dx = screenX - translateX
        val dy = screenY - translateY
        out[0] = (m11 * dx - m01 * dy) * inverseDeterminant
        out[1] = (-m10 * dx + m00 * dy) * inverseDeterminant
    }

    fun zoomAround(anchorX: Float, anchorY: Float, newScale: Float): ViewTransform {
        val safeScale = newScale.coerceIn(MIN_SCALE, MAX_SCALE)
        val factor = safeScale / scale
        return copy(
            scale = safeScale,
            translateX = anchorX - (anchorX - translateX) * factor,
            translateY = anchorY - (anchorY - translateY) * factor,
        )
    }

    fun rotateAround(anchorX: Float, anchorY: Float, deltaRad: Float): ViewTransform {
        val c = cos(deltaRad)
        val s = sin(deltaRad)
        val dx = translateX - anchorX
        val dy = translateY - anchorY
        return copy(
            rotationRad = rotationRad + deltaRad,
            translateX = anchorX + dx * c - dy * s,
            translateY = anchorY + dx * s + dy * c,
        )
    }

    /** Builds a column-major matrix for canvas to OpenGL clip coordinates. */
    fun buildCanvasToClip(viewWidth: Float, viewHeight: Float, out: FloatArray) {
        require(out.size >= 16)
        require(viewWidth > 0f)
        require(viewHeight > 0f)
        val sx = 2f / viewWidth
        val sy = -2f / viewHeight
        out[0] = m00 * sx; out[1] = m10 * sy; out[2] = 0f; out[3] = 0f
        out[4] = m01 * sx; out[5] = m11 * sy; out[6] = 0f; out[7] = 0f
        out[8] = 0f; out[9] = 0f; out[10] = 1f; out[11] = 0f
        out[12] = translateX * sx - 1f; out[13] = translateY * sy + 1f
        out[14] = 0f; out[15] = 1f
    }

    companion object {
        const val MIN_SCALE = 0.02f
        const val MAX_SCALE = 64f
    }
}
```

app/src/main/java/com/wetinknext/engine/gl/CanvasGeometry.kt

```kotlin
package com.wetinknext.engine.gl

import android.opengl.GLES30
import java.nio.ByteBuffer
import java.nio.ByteOrder

/** A document-sized quad; GL resources are owned by the render thread. */
class CanvasGeometry {
    private var vaoId = 0
    private var vboId = 0

    fun create(width: Int, height: Int) {
        release()
        val vertices = floatArrayOf(0f, 0f, width.toFloat(), 0f, 0f, height.toFloat(), width.toFloat(), height.toFloat())
        val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
        data.put(vertices).position(0)
        val ids = IntArray(1)
        GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
        GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
        GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
        GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
        GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
        GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
    }

    fun draw() {
        check(vaoId != 0)
        GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4); GLES30.glBindVertexArray(0)
    }

    fun release() {
        if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0)
        if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
        vaoId = 0; vboId = 0
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30
import java.nio.ByteBuffer
import java.nio.ByteOrder

/** A single cached fullscreen triangle for future present passes. */
class GeometryCache {
    private var vaoId = 0
    private var vboId = 0
    fun create() {
        if (vaoId != 0) return
        val vertices = floatArrayOf(-1f, -1f, 3f, -1f, -1f, 3f)
        val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
        data.put(vertices).position(0)
        val ids = IntArray(1); GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
        GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
        GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
        GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
        GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
        GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
    }
    fun drawFullscreenTriangle() { check(vaoId != 0); GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLES, 0, 3); GLES30.glBindVertexArray(0) }
    fun release() { if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0); if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0); vboId = 0; vaoId = 0 }
}
```

```kotlin
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Capabilities queried only after a GLES 3 context is current. */
data class GlCaps(
    val renderer: String,
    val version: String,
    val supportsHalfFloatColorBuffer: Boolean,
) {
    companion object {
        fun query(): GlCaps {
            val extensions = GLES30.glGetString(GLES30.GL_EXTENSIONS).orEmpty()
            return GlCaps(
                renderer = GLES30.glGetString(GLES30.GL_RENDERER).orEmpty(),
                version = GLES30.glGetString(GLES30.GL_VERSION).orEmpty(),
                supportsHalfFloatColorBuffer = extensions.contains("EXT_color_buffer_half_float"),
            )
        }
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** GL error checks intended for setup boundaries, never the render hot path. */
object GlCheck {
    fun noError(label: String) {
        val error = GLES30.glGetError()
        check(error == GLES30.GL_NO_ERROR) { "$label: GL error 0x${error.toString(16)}" }
    }

    fun framebufferComplete(label: String) {
        val status = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER)
        check(status == GLES30.GL_FRAMEBUFFER_COMPLETE) {
            "$label: framebuffer incomplete (0x${status.toString(16)})"
        }
    }
}
```

```kotlin
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Linked GLSL program. Construction and release must run on the GL thread. */
class GlProgram(vertexSource: String, fragmentSource: String) {
    val id: Int = GLES30.glCreateProgram().also { program ->
        check(program != 0) { "Unable to create GL program" }
        val vertex = compile(GLES30.GL_VERTEX_SHADER, vertexSource)
        val fragment = compile(GLES30.GL_FRAGMENT_SHADER, fragmentSource)
        GLES30.glAttachShader(program, vertex); GLES30.glAttachShader(program, fragment); GLES30.glLinkProgram(program)
        val linked = IntArray(1); GLES30.glGetProgramiv(program, GLES30.GL_LINK_STATUS, linked, 0)
        GLES30.glDeleteShader(vertex); GLES30.glDeleteShader(fragment)
        check(linked[0] == GLES30.GL_TRUE) { "Program link failed: ${GLES30.glGetProgramInfoLog(program)}" }
    }
    fun use() = GLES30.glUseProgram(id)
    fun release() = GLES30.glDeleteProgram(id)

    private fun compile(type: Int, source: String): Int = GLES30.glCreateShader(type).also { shader ->
        check(shader != 0) { "Unable to create shader" }
        GLES30.glShaderSource(shader, source); GLES30.glCompileShader(shader)
        val compiled = IntArray(1); GLES30.glGetShaderiv(shader, GLES30.GL_COMPILE_STATUS, compiled, 0)
        check(compiled[0] == GLES30.GL_TRUE) { "Shader compile failed: ${GLES30.glGetShaderInfoLog(shader)}" }
    }
}
```

app/src/main/java/com/wetinknext/engine/gl/RenderTarget.kt

```kotlin
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Texture-backed FBO with RGBA16F probing and a guaranteed RGBA8 fallback. */
class RenderTarget {
    var framebufferId: Int = 0
        private set
    var textureId: Int = 0
        private set
    var width: Int = 0
        private set
    var height: Int = 0
        private set
    var usesHalfFloat: Boolean = false
        private set

    fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
        require(width > 0 && height > 0)
        release()
        if (preferHalfFloat && createInternal(width, height, true)) return
        check(createInternal(width, height, false)) { "RGBA8 render target creation failed" }
    }

    /** A 4x4 completeness probe used before allocating a document-sized target. */
    fun probeHalfFloatColorBuffer(): Boolean {
        release()
        val result = createInternal(4, 4, true)
        release()
        return result
    }

    fun bind() {
        check(framebufferId != 0) { "RenderTarget is not created" }
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebufferId)
    }

    fun clear(red: Float, green: Float, blue: Float, alpha: Float) {
        bind()
        GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
        GLES30.glDisable(GLES30.GL_BLEND)
        GLES30.glClearColor(red, green, blue, alpha)
        GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
    }

    fun release() {
        if (framebufferId != 0) GLES30.glDeleteFramebuffers(1, intArrayOf(framebufferId), 0)
        if (textureId != 0) GLES30.glDeleteTextures(1, intArrayOf(textureId), 0)
        framebufferId = 0; textureId = 0; width = 0; height = 0; usesHalfFloat = false
    }

    private fun createInternal(targetWidth: Int, targetHeight: Int, halfFloat: Boolean): Boolean {
        val textures = IntArray(1)
        val framebuffers = IntArray(1)
        GLES30.glGenTextures(1, textures, 0)
        GLES30.glGenFramebuffers(1, framebuffers, 0)
        val texture = textures[0]
        val framebuffer = framebuffers[0]
        if (texture == 0 || framebuffer == 0) return false
        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texture)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MIN_FILTER, GLES30.GL_LINEAR)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MAG_FILTER, GLES30.GL_LINEAR)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_S, GLES30.GL_CLAMP_TO_EDGE)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_T, GLES30.GL_CLAMP_TO_EDGE)
        GLES30.glTexImage2D(GLES30.GL_TEXTURE_2D, 0, if (halfFloat) GLES30.GL_RGBA16F else GLES30.GL_RGBA8,
            targetWidth, targetHeight, 0, GLES30.GL_RGBA, if (halfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE, null)
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebuffer)
        GLES30.glFramebufferTexture2D(GLES30.GL_FRAMEBUFFER, GLES30.GL_COLOR_ATTACHMENT0, GLES30.GL_TEXTURE_2D, texture, 0)
        val complete = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER) == GLES30.GL_FRAMEBUFFER_COMPLETE
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
        if (!complete) {
            GLES30.glDeleteFramebuffers(1, framebuffers, 0); GLES30.glDeleteTextures(1, textures, 0)
            return false
        }
        framebufferId = framebuffer; textureId = texture; width = targetWidth; height = targetHeight; usesHalfFloat = halfFloat
        return true
    }
}
```

```kotlin
package com.wetinknext.engine.gl

object ShaderLib {
    const val canvasPresentVertex = """#version 300 es
        layout(location = 0) in vec2 aPosition;
        uniform mat4 uCanvasToClip;
        uniform vec2 uCanvasSize;
        out vec2 vUv;
        void main() {
            vUv = aPosition / uCanvasSize;
            gl_Position = uCanvasToClip * vec4(aPosition, 0.0, 1.0);
        }
    """
    const val presentFragment = """#version 300 es
        precision highp float;
        in vec2 vUv;
        uniform sampler2D uTexture;
        out vec4 fragColor;
        void main() { fragColor = texture(uTexture, vUv); }
    """
    const val fullscreenVertex = """#version 300 es
        layout(location = 0) in vec2 aPosition;
        out vec2 vUv;
        void main() { vUv = aPosition * 0.5 + 0.5; gl_Position = vec4(aPosition, 0.0, 1.0); }
    """
    const val solidFragment = """#version 300 es
        precision highp float;
        in vec2 vUv;
        out vec4 fragColor;
        void main() { fragColor = vec4(vUv, 0.0, 1.0); }
    """
}
```

```kotlin
package com.wetinknext.engine.input

enum class InputAction {
    DOWN,
    MOVE,
    UP,
    CANCEL,
}
```

```kotlin
package com.wetinknext.engine.input

/** Fixed-capacity container for passing input from the UI thread to the GL thread. */
class InputBatch(val maxSamples: Int) {
    val samples = Array(maxSamples) { InputSample() }

    var action: InputAction = InputAction.MOVE
        private set
    var sampleCount: Int = 0
        private set
    var prediction: Boolean = false
        private set

    fun begin(action: InputAction, prediction: Boolean = false) {
        this.action = action
        this.prediction = prediction
        sampleCount = 0
    }

    fun addSample(
        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
        historical: Boolean,
    ): Boolean {
        if (sampleCount >= maxSamples) return false
        samples[sampleCount].set(
            canvasX, canvasY, pressure.coerceIn(0f, 1f), tiltX.coerceIn(-1f, 1f),
            tiltY.coerceIn(-1f, 1f), orientationRad, timestampNanos, pointerId, tool, historical,
        )
        sampleCount += 1
        return true
    }

    fun isEmpty(): Boolean = sampleCount == 0

    fun clear() {
        for (index in 0 until sampleCount) samples[index].clear()
        sampleCount = 0
        prediction = false
        action = InputAction.MOVE
    }
}
```

```kotlin
package com.wetinknext.engine.input

import java.util.concurrent.atomic.AtomicInteger
import java.util.concurrent.atomic.AtomicReferenceArray

/** Fixed-size pool that returns null instead of allocating when exhausted. */
class InputBatchPool(
    batchCount: Int = DEFAULT_BATCH_COUNT,
    maxSamplesPerBatch: Int = DEFAULT_MAX_SAMPLES,
) {
    private val batches = AtomicReferenceArray<InputBatch>(batchCount)
    private val available = AtomicInteger(batchCount)

    init {
        require(batchCount > 0)
        require(maxSamplesPerBatch > 0)
        for (index in 0 until batchCount) batches.set(index, InputBatch(maxSamplesPerBatch))
    }

    fun acquire(): InputBatch? {
        while (true) {
            val current = available.get()
            if (current <= 0) return null
            if (available.compareAndSet(current, current - 1)) {
                for (index in 0 until batches.length()) {
                    val batch = batches.getAndSet(index, null)
                    if (batch != null) return batch
                }
                available.incrementAndGet()
                return null
            }
        }
    }

    fun release(batch: InputBatch) {
        batch.clear()
        for (index in 0 until batches.length()) {
            if (batches.compareAndSet(index, null, batch)) {
                available.incrementAndGet()
                return
            }
        }
        error("InputBatchPool overflow: released batch does not belong to pool")
    }

    val freeCount: Int get() = available.get()

    companion object {
        const val DEFAULT_BATCH_COUNT = 96
        const val DEFAULT_MAX_SAMPLES = 64
    }
}
```

```kotlin
package com.wetinknext.engine.input

/** Mutable input sample allocated only while a batch pool is initialized. */
class InputSample {
    var canvasX = 0f
    var canvasY = 0f
    var pressure = 0f
    var tiltX = 0f
    var tiltY = 0f
    var orientationRad = 0f
    var timestampNanos = 0L
    var pointerId = -1
    var tool = PointerTool.UNKNOWN
    var historical = false

    fun set(
        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
        historical: Boolean,
    ) {
        this.canvasX = canvasX; this.canvasY = canvasY; this.pressure = pressure
        this.tiltX = tiltX; this.tiltY = tiltY; this.orientationRad = orientationRad
        this.timestampNanos = timestampNanos; this.pointerId = pointerId; this.tool = tool
        this.historical = historical
    }

    fun clear() {
        canvasX = 0f; canvasY = 0f; pressure = 0f; tiltX = 0f; tiltY = 0f
        orientationRad = 0f; timestampNanos = 0L; pointerId = -1
        tool = PointerTool.UNKNOWN; historical = false
    }
}
```

```
package com.wetinknext.engine.input

enum class PointerTool {
    FINGER,
    STYLUS,
    ERASER,
    UNKNOWN,
}
```

```kotlin
package com.wetinknext.ui.theme

import androidx.compose.ui.graphics.Color

data class WetInkNextPalette(
    val appBg: Color,
    val panelBg: Color,
    val panelStroke: Color,
    val textPrimary: Color,
    val textSecondary: Color,
    val accent: Color,
) {
    companion object {
        val default = WetInkNextPalette(
            appBg = Color(0xFF17181C),
            panelBg = Color(0xFF25272D),
            panelStroke = Color(0xFF3A3D46),
            textPrimary = Color(0xFFF3F4F6),
            textSecondary = Color(0xFFA4A8B3),
            accent = Color(0xFF4D7AFF),
        )
    }
}
```

```
package com.wetinknext.engine.core

import org.junit.Assert.assertArrayEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class DirtyRectTest {
    @Test fun includesDabsAndConvertsToPixelBounds() {
        val rect = DirtyRect()
        rect.include(100f, 120f, 10f)
        rect.include(150f, 140f, 5f)
        val pixels = IntArray(4)
        rect.toPixelBounds(pixels)
        assertArrayEquals(intArrayOf(90, 110, 155, 145), pixels)
    }

    @Test fun clampCanMakeRectEmpty() {
        val rect = DirtyRect()
        rect.include(-100f, -100f, -10f)
        rect.clamp(100f, 100f)
        assertTrue(rect.isEmpty)
    }

    @Test fun clearResetsRect() {
        val rect = DirtyRect()
        rect.include(0f, 0f, 10f)
        rect.clear()
        assertTrue(rect.isEmpty)
    }
}
```

```
package com.wetinknext.engine.core

import org.junit.Assert.assertEquals
import org.junit.Test

class ViewTransformTest {
    @Test fun canvasAndScreenCoordinatesAreInverse() {
        val transform = ViewTransform(120f, 80f, 2.5f, 0.35f, true)
        val screen = FloatArray(2)
        val canvas = FloatArray(2)
        transform.canvasToScreen(240f, 310f, screen)
        transform.screenToCanvas(screen[0], screen[1], canvas)
        assertEquals(240f, canvas[0], 0.001f)
        assertEquals(310f, canvas[1], 0.001f)
    }

    @Test fun zoomAroundAnchorKeepsAnchorStable() {
        val initial = ViewTransform(translateX = 20f, translateY = 30f)
        val before = FloatArray(2)
        val after = FloatArray(2)
        initial.screenToCanvas(400f, 300f, before)
        initial.zoomAround(400f, 300f, 3f).screenToCanvas(400f, 300f, after)
        assertEquals(before[0], after[0], 0.001f)
        assertEquals(before[1], after[1], 0.001f)
    }
}
```